

1. 架构设计反思

Lab 1 — Shell

- Shell 的核心是 `handle_line()` 函数，整体分为三个阶段：解析、校验、执行。
 - **解析阶段**：首先用 `strchr` 在输入行中查找 `|`，将其分割为多个命令段，再对每段用 `strtok_r` 按空白符分词，构造 `argv` 数组。选择先分割管道符再分词的顺序，是因为这样可以天然处理 `echo hello|cat`（无空格管道符）的情况，不需要额外处理。（但是没有解析引号 按照 handout 是这样的）
 - **校验阶段**：在 fork 任何进程之前，先验证整个 pipeline 的合法性，包括空段检测（leading/trailing/doubled pipe）、参数数量上限、builtin 在 pipeline 中的拒绝。提前校验的好处是：一旦发现错误，不需要清理任何子进程或 pipe fd，错误处理路径更简单。
 - **执行阶段**：对于 N 个命令，预先创建 N-1 个 pipe，再依次 fork。每个子进程用 `dup2` 将 stdin/stdout 重定向到对应 pipe 端，然后关闭所有 pipe fd，再调用 `execvp`。父进程在所有子进程 fork 完成后，统一关闭所有 pipe fd，再依次 `waitpid`。（可以画出几个 fork 出来的图更加容易理解的）
- 这里有一个关键细节也就是说父进程必须在 fork 完所有子进程之后才关闭 pipe fd。如果在 fork 中途就关闭，后续子进程可能继承到已关闭的 fd，导致管道断裂。
- **Builtin 处理**：`cd` 和 `exit` 必须在 shell 进程本身执行，而不能 fork 子进程。原因是 `chdir()` 只改变当前进程的工作目录，如果在子进程里调用，shell 自身的目录不会变化，`cd` 就失去了意义。

Lab 2 — LLM Client

- 整体流程是标准的 TCP 客户端模式：`getaddrinfo` → `socket` → `connect` → `send` → `recv` → `close`。
 - **构造请求**：采用“先构造 body，再构造完整请求”的两步策略。先用 `snprintf` 将 JSON body 写入临时缓冲区并获取其精确字节数，再将这个长度填入 `Content-Length` 头部。这样避免了需要先预测 body 长度的问题，也保证了 `Content-Length` 的准确性。
 - **发送数据**：`send_all()` 实现了循环发送，处理了 TCP 的字节流特性——单次 `send()` 调用不保证发送全部数据。同时处理了 `EINTR`（被信号中断）的情况，保证鲁棒性。
 - **接收数据**：`recv_response()` 循环读取直到 `recv()` 返回 0（对端关闭连接）。由于请求头中携带了 `Connection: close`，服务器响应完成后会主动关闭连接，因此不需要解析 `Content-Length` 来判断响应结束，简化了实现。
 - **解析响应**：用 `strstr` 找到 `HTTP/1.1` 后读取状态码，用 `strstr` 找到 `\r\n\r\n` 定位 body 起始位置。选择在接收完整响应后再解析，而不是边收边解析，避免了 `\r\n\r\n` 跨越两次 `recv()` 调用的边界问题。

Lab 3 — Thread Pool

- **状态设计**：共享状态包括：任务队列 `tasks`、当前正在执行的任务数 `executing`、shutdown 标志 `shutdown`，以及两个 condition variable。

- 选择使用两个 condition variable 而非一个：
 - `cond_work`: worker 等待新任务时使用，由 `submit()` signal
 - `cond_idle`: `wait()` 等待池空闲时使用，由 worker 完成任务后 broadcast

这样分离的好处是语义清晰，`wait()` 不会被 `submit()` 的 signal 误唤醒，减少不必要的竞争和重新检查。

- 阶段实现
 - **Worker**: worker 持锁检查条件，用 `while` 包裹 `pthread_cond_wait` 防止虚假唤醒，取出任务后**释放锁再执行**，执行完成后重新持锁更新 `executing` 计数并通知 `cond_idle`。释放锁后执行任务是关键设计：一方面保证真正的并行执行，另一方面 允许任务内部调用 `submit()`（可重入提交）而不死锁。
 - **Wait**: `thread_pool_wait()` 等待条件是 `tasks.count == 0 && executing == 0`，两者缺一不可。仅检查队列为空是不够的——worker 可能已经取出任务但尚未执行完成，此时队列为空但 `executing > 0`，pool 并未真正空闲。
 - **Destroy**: 设置 `shutdown = 1` 后 broadcast 唤醒所有 sleeping worker，worker 检查到 shutdown 后退出循环，父线程 依次 join 所有 worker，最后清理队列中未执行的任务并释放资源。已经在执行中的任务会自然跑完（worker 在取任务后才检查 shutdown）

2. 时间估计 vs 实际花费

Lab	预估时间	实际花费
Lab 1 Shell	2 小时	2 小时
Lab 2 LLM Client	2 小时	2 小时
Lab 3 Thread Pool	2 小时	5 小时

3. 遇到的挑战

Lab 1 - pipe fd 的关闭时机

最初实现时，在 fork 每个子进程后立即在父进程关闭该子进程 不需要的 pipe fd。这导致后续子进程 fork 时继承的 fd 表不完整， 多级管道出现读端提前 EOF 的问题。

解决方法：将父进程关闭 pipe fd 的操作统一移到所有子进程 fork 完成之后，保证每个子进程继承到完整的 pipe fd 表，再由子进程自己关闭不需要的端。

Lab 2 - 测试caddy的奇怪报错

最初按照handout给出的`caddy reverse-proxy --from :18080 --to https://models.sjtu.edu.cn` 会得不到回答响应，经过Minimax的思考找到了一个解决方案。

```
caddy reverse-proxy --from :18080 /
--to https://models.sjtu.edu.cn --change-host-header
# - Caddy需要--change-host-header (把 Host 头改成 upstream 的域名)
```

Lab 3

- TSan 环境不兼容

在虚拟机上运行 `make tsan` 时遇到：

```
FATAL: ThreadSanitizer: unexpected memory mapping
```

原因是 TSan 与当前内核版本的内存映射格式不兼容。通过 `sudo sysctl vm.mmap_rnd_bits=28` 降低 ASLR 随机化位数后解决。

```
之前没仔细看TA WeiWang 发在群里的消息
```

- 代码设计

在lab3里面可能是因为文件数目多，也更加困难所以才觉得困难了。主要耗时在理解 `wait()` 的正确语义上——最初只检查队列为空，忽略了 `executing` 计数器，导致 `wait()` 提前返回

4. 对课程的建议

- 希望在课上讲一下tsan的机制，才会更加理解设计的时候的东西的
- Lab 2 的 socket api 可以给更多参考代码